Přírodovědecká fakulta

Univerzita Karlova

Algoritmy počítačové kartografie

Generalizace budov LOD0

Kristýna Antošová, Vít Kadlec

1.N-GKDPZ

Praha 2026

1 Zadání

Vstup: množina budov $B = \{B_i\}_{i=}^{n}$, *budova* $B_i = \{P_{i,j}\}_{j=1}^{m}$.

Výstup: $G(B_i)$.

Ze souboru načtěte vstupní data představovaná lomovými body budov a proveďte generalizaci budov do úrovně detailu LOD0. Pro tyto účely použijte vhodnou datovou sadu, např. ZABAGED, testování proveďte nad třemi datovými sadami (historické centrum města, sídliště, izolovaná zástavba).

Pro každou budovu určete její hlavní směry metodami:

- Minimum Area Enclosing Rectangle,
- PCA.

U první metody použijte některý z algoritmů pro konstrukci konvexní obálky. Budovu při generalizaci do úrovně LOD0 nahraďte obdélníkem orientovaným v obou hlavních směrech, se středem v těžišti budovy, jeho plocha bude stejná jako plocha budovy. Výsledky generalizace vhodně vizualizujte.

Otestujte a porovnejte efektivitu obou metod s využitím hodnotících kritérií. Pokuste se rozhodnout, pro které tvary budov dávají metody nevhodné výsledky, a pro které naopak poskytují vhodnou aproximaci.

Hodnocení:

Krok	**Hodnocení**
Generalizace budov metodami Minimum Area Enclosing Rectangle a PCA.	15b
Generalizace budov metodou Longest Edge.	*+5b*
Generalizace budov metodou Wall Average.	*+8b*
Generalizace budov metodou Weighted Bisector.	*+10b*
Implementace další metody konstrukce konvexní obálky.	*+5b*
Ošetření singulárních případů při generování konvexní obálky.	*+2b*
*Načtení vstupních dat ze *.shp.*	*+10b*
Max celkem:	**55b**

V rámci zadání byly řešeny všechny výše zmíněné bonusové úlohy.

2 Popis a rozbor problému

2.1 Generalizace budov

Generalizací se rozumí proces, jehož cílem je zjednodušit jednotlivé objekty v závislosti na velikosti měřítka mapy. Se zmenšením měřítka (oddálením mapy) tedy budou zakresleny objekty méně detailněji. Generalizace budov zahrnuje různé postupy, mezi něž patří zjednodušení geometrie, seskupování blízkých objektů, výběr zobrazovaných objektů, změna velikosti a polohy, či zachování charakteristických rysů. Tyto metody jsou vždy prováněny tak, aby byly zachovány topologické vztahy objektů.

K automatizaci generalizace jsou používány nejčastěji algoritmy *Minimum Area Enclosing/Bounding Rectangle*, *Principal Component Analysis*, *Longest Edge*, *Weighted Bisector* nebo *Wall Average*. Všechny tyto algoritmy jsou také využity v rámci řešení této úlohy, kdy jsou budovy generalizovány do úrovně LOD0. Každá budova je nahrazena obdélníkem orientovaným v obou hlavních směrech, se středem v těžišti budovy. Plocha výsledného obdélníku je přitom shodná s plochou původní budovy. Výsledné vrcholy V_i' obdélníku jsou spočteny pomocí těžiště T, směrových vektorů u_i a poměru ploch k:

$$V_i' = T + u_i' = T + \sqrt{k}\, u_i, \quad \text{kde} \quad k = \frac{A_b}{A}, \quad u_i = V_i - T. \tag{1}$$

2.2 Konvexní obálka

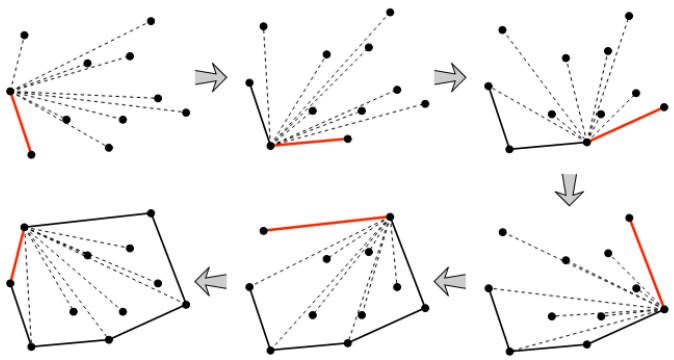
Konvexní obálka je taková množina bodů, která je minimální a všechny body leží uvnitř tohoto konvexního mnohoúhelníku. Zároveň platí, že všechny vnitřní úhly konvexního mnohoúhelníku nejsou větší než 180°. Konvexní obálka slouží jako pomocná struktura algoritmu Minimum Area Bounding Rectangle. Metody pro konstrukci konvexní obálky jsou například *Jarvis Scan*, *Graham Scan*, *Quick Hull* nebo *Divide and Conquer*.

2.2.1 Jarvis Scan

Jarvis Scan (obrázek 1) je metoda konstrukce konvexní obálky vhodná pro množiny bodů menších velikostí s časovou složitostí $O(n \cdot h)$. Algoritmus vezme první bod q (pivot) o minimální y-souřadnici, ležící na konvexní obálce. Následně jsou do konvexní obálky přidávány body, které svírají s poslední dvojicí bodů v obálce největší úhel ω:

$$p_{j+1} = \arg\max_{\forall p_i \in P} \angle(p_{j-1}, p_j, p_i) \tag{2}$$

Tentoto proces je opakován do té doby, než je dosaženo počátečního bodu q.



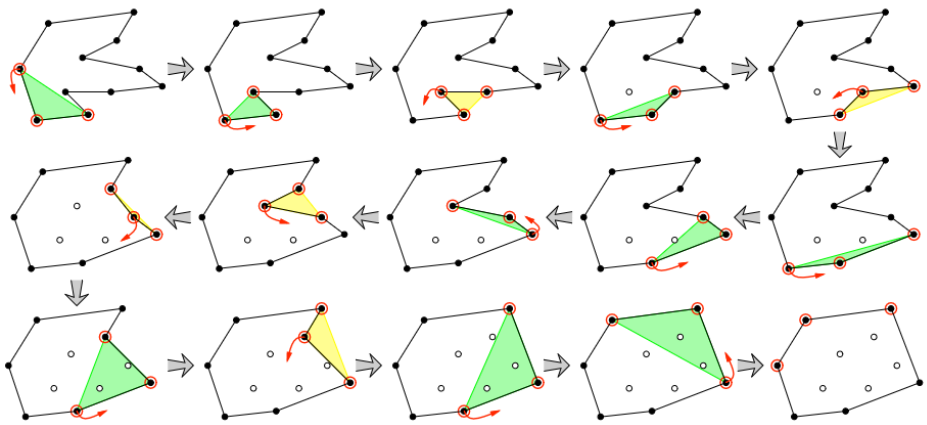
Obrázek 1: *Jarvis Scan (zdroj: Erickson 2008)*

2.2.2 Graham Scan

Graham Scan je oproti předchozí metodě vhodný pro větší množiny bodů, díky jeho časové složitosti $O(n \cdot \log n)$. Nejprve je stejným způsobem jako u metody Jarvis Scan nalezen pivot. Poté jsou všechny body seřazeny vzestupně podle velikosti úhlu ω, což je úhel, který svírají s přímkou procházející pivotem a rovnoběžnou s osou x. Následně je u seřazených bodů vyhodnocováno kritérium levotočivosti:

$$p_j \begin{cases} \notin \mathcal{H}, & p_{j+1} \in \sigma_r(p_{j-1}, p_j), \\ ? \in \mathcal{H}, & p_{j+1} \in \sigma_l(p_{j-1}, p_j). \end{cases} \tag{3}$$

Pokud se následující bod nachází v pravé polorovině od přímky procházející posledními dvěma body, je do konvexní obálky přidán právě procházený bod. Poslední bod je naopak z konvexní obálky vyjmut, jak je vidět na obrázku 2.



Obrázek 2: *Graham Scan (zdroj: Erickson 2008)*

2.3 Minimum Area Bounding Rectangle

Tento algoritmus je primárně určen pro detekci natočení budovy. Nejdříve je kolem množiny bodů P vytvořena konvexní obálka, která je následně rotována o úhel $-\sigma$ tak, aby byla postupně každá hrana rovnoběžná s osou x:

$$P_0 = R(-\sigma)P \Rightarrow \begin{bmatrix} x_r \\ y_r \end{bmatrix} = \begin{bmatrix} \cos(-\sigma) & -\sin(-\sigma) \\ \sin(-\sigma) & \cos(-\sigma) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \tag{4}$$

Zároveň je po každém takovém otočení spočtena plocha obdélníku. Jako konečný výsledek je brán obdélník s nejmenší plochou. Nakonec je tento obdélník je otočen zpět o úhel σ. Stejný princip rotace je také využit u zbylých zprcovaných algoritmů generalizace.

2.4 Principal Component Analysis

Metoda hlavních komponent (PCA) využívá singulární rozklad kovarianční matice C:

$$C = U\Sigma V^T, \quad C = \begin{bmatrix} C(A,A) & C(A,B) \\ C(B,A) & C(B,B) \end{bmatrix} \tag{5}$$

kde U a V jsou ortogonální matice vlastních vektorů a Σ je diagonální matice se čtverci velikostí vlastních vektorů. Hlavní směr σ budovy je dán směrem prvního vlastního vektoru.

2.5 Longest Edge

Metoda Longest Edge předpokládá, že hlavní směr budovy je určen její nejdelší hranou. Druhý hlavní směr je na něj kolmý. Jedná se o nejjednodušší přístup k detekci hlavního směru, který však nemusí poskytovat dobré výsledky u budov s nepravidelným půdorysem, kde nejdelší hrana nemusí odpovídat skutečné orientaci budovy. Směrnice σ nejdelší hrany je vypočtena z jejích koncových bodů a, b:

$$\sigma = \operatorname{atan2}(b_y - a_y,\ b_x - a_x) \tag{6}$$

Budova je následně otočena o úhel $-\sigma$, kolem takto natočené budovy je vytvořen Min-max box a ten je otočen zpět o úhel σ.

2.6 Wall Average

Tato metoda funguje nejlépe za předpokladu pravoúhlých hran budov. Pro každý vrchol se spočítá vnitřní úhel ω_i a z něj orientovaný zbytek po dělení $\frac{\pi}{2}$:

$$k_i = \frac{2\omega_i}{\pi}, \quad r_i = (k_i - \lfloor k_i \rfloor)\frac{\pi}{2} \tag{7}$$

Pokud $r_i > \frac{\pi}{4}$, provedeme korekci $r_i = r_i - \frac{\pi}{2}$. Hlavní směr budovy se spočítá jako vážený průměr zbytků, kde vahou jsou délky stran:

$$\sigma = \sigma_{1,2} + \sum_{i=1}^{n} \frac{r_i s_i}{s_i} \tag{8}$$

2.7 Weighted Bisector

Metoda Weighted Bisector hledá dvě nejdelší úhlopříčky polygonu budovy. Tyto úhlopříčky musí být spojnicemi dvou vrcholů polygonu a nesmí protínat žádnou z hran polygonu, což je ověřeno determinantovým testem. Hlavní směr σ se spočítá váženým průměrem směrnic dvou nejdelších úhlopříček, kde vahami jsou jejich délky d_1 a d_2:

$$\sigma = \frac{d_1\sigma_1 + d_2\sigma_2}{d_1 + d_2} \tag{9}$$

2.8 Hodnocení efektivity

Pro hodnocení efektivity detekce hlavních směrů byla použita střední hodnota čtverců úhlových odchylek jednotlivých segmentů:

$$\Delta\sigma = \frac{\pi}{2n}\sqrt{\sum_{i=1}^{n}(r_i - r)^2} \tag{10}$$

V kartografické praxi je běžná akceptace objektů s odchylkou $\Delta\sigma < 10°$.

3 Popisy algoritmů formálním jazykem

3.1 Jarvis Scan

Algorithm 1 Jarvis Scan

Input: polygon *pol*
Output: convex hull *ch*

```
remove duplicate points from pol
if |pol| < 3 or points are collinear then return empty polygon
end if
find pivot q with minimum y-coordinate (if tie, minimum x)
find point s with minimum x-coordinate
p_j ← q, p_{j-1} ← (s_x, q_y), add p_j to ch
while true do
    ω_max ← 0
    for each vertex p_i ∈ pol, p_i ≠ p_j do
        compute ω = ∠(p_{j-1}, p_j, p_i)
        if ω > ω_max then update ω_max and index
        end if
    end for
    add point with ω_max to ch, p_{j-1} ← p_j, p_j ← new point
    if p_j = q then break
    end if
end while
```

3.2 Graham Scan

Algorithm 2 Graham Scan

Input: polygon *pol*
Output: convex hull *ch*

```
remove duplicate points from pol
if |pol| < 3 or points are collinear then return empty polygon
end if
find pivot q with minimum y-coordinate
add q to ch
for each vertex p_i ≠ q do
    compute angle ω relative to horizontal vector from q
    if same ω, keep the point farther from q
end for
sort points by ω ascending, add first to ch
for remaining sorted points p_j do
    while |ch| > 1 and p_j lies to the right of last 2 points in ch do
        remove last point from ch
    end while
    add p_j to ch
end for
```

3.3 Minimum Area Bounding Rectangle

Algorithm 3 Minimum Area Bounding Rectangle

Input: polygon *building*
Output: minimum bounding rectangle *mbr*

```
create convex hull ch (Jarvis or Graham)
initialize MMB_min and area_min
for each edge of convex hull do
    compute direction σ
    rotate ch by −σ, construct MMB, compute area
    if area < area_min then update area_min, MMB_min, σ_min
    end if
end for
rotate MMB_min back by σ_min
resize area to match building area
```

3.4 PCA

Algorithm 4 Principal Component Analysis

Input: polygon *building*
Output: generalized rectangle *mbr*

```
create matrix A from point coordinates
compute covariance matrix C, perform SVD decomposition
compute direction σ from eigenvectors
rotate building by −σ, compute MMB
rotate MMB back by σ
resize area to match building area
```

3.5 Longest Edge

Algorithm 5 Longest Edge

Input: polygon *building*
Output: generalized rectangle *mbr*

```
longest ← 0
for each edge of polygon do
    if edge length > longest then update longest and endpoints a, b
    end if
end for
σ ← atan2(b_y − a_y, b_x − a_x)
rotate building by −σ, compute MMB, rotate back by σ
resize area to match building area
```

3.6 Wall Average

Algorithm 6 Wall Average

Input: polygon *building*
Output: generalized rectangle *mbr*

```
compute directions σ_i of all edges
σ_ref ← σ_0 (direction of first edge)
for each vertex i do
    ω_i ← |σ_i − σ_{i−1}|
    k_i ← 2ω_i/π, r_i ← (k_i − ⌊k_i⌋) · π/2
    if r_i > π/4 then r_i ← r_i − π/2
    end if
    s_i ← edge length from vertex i
end for
σ ← σ_ref + Σ r_i s_i / Σ s_i
rotate, MMB, back rotation, resize
```

3.7 Weighted Bisector

Algorithm 7 Weighted Bisector

Input: polygon *building* (≥ 4 vertices)
Output: generalized rectangle *mbr*

```
find all diagonals, sort descending by length
for each diagonal do
    if does not intersect any edge (determinant test) then
        store length d_k and direction σ_k
    end if
    if 2 valid diagonals found then
        σ ← (d_1 σ_1 + d_2 σ_2)/(d_1 + d_2)
        rotate, MMB, back rotation, resize
    end if
end for
```

4 Vstupní data a Aplikace

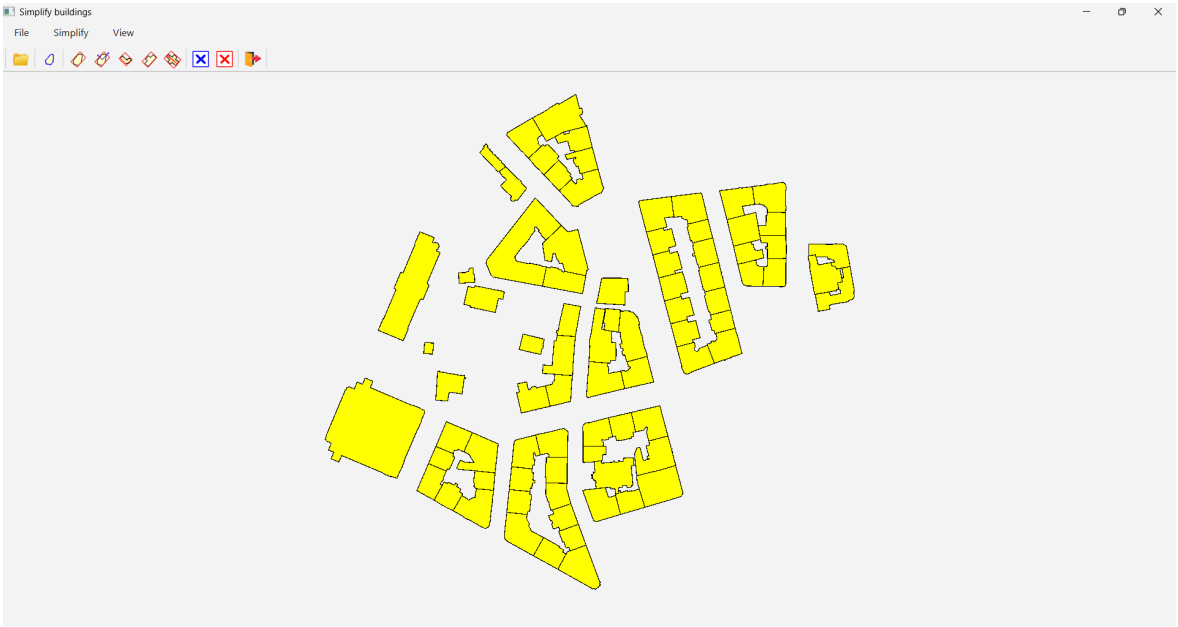
Vstupními daty jsou polygonové vrstvy budov ve formátu shapefile, získané z datové sady ČÚZK (katastrální mapa). Testování bylo provedeno nad třemi datovými sadami reprezentujícími různé typy zástavby:

- **Josefov** – historické centrum s nepravidelnými půdorysy budov,
- **Kročehlavy** – sídlištní zástavba s pravidelnými tvary,
- **Suchdol** – izolovaná zástavba s rozptýlenými budovami.

Geografické souřadnice jsou při načtení převedeny do souřadnic obrazovky s uniformním měřítkem, které zachovává poměr stran vstupních dat bez ohledu na rozměry okna aplikace. Data jsou vycentrována s okraji 20 px.

Po spuštění skriptu *MainForm.py* se otevře okno aplikace. V záložce *File* jsou umístěny funkce *Open* a *Exit*. V záložce *Simplify* jsou algoritmické funkce *MBR*, *PCA*, *Longest Edge*, *Wall Average*, *Weighted Bisector* a funkce na přepínání metody tvorby konvexní obálky *Jarvis/Graham Scan*. Poslední záložka *View* obsahuje funkce *Clear results* a *Clear all*. Všechny funkce mají ikonku nacházející se na nástrojové liště. Aplikaci lze ukončit funkcí *Exit* nebo tlačítkem v pravém horním rohu.

Po kliknutí na funkci *Open* se otevře nabídka pro vybrání požadovaného souboru ve formátu shapefile. V aplikaci lze též kreslit svůj vlastní polygon, jestliže není otevřen shapefile soubor. Kliknutím na funkci *Clear results* se smaže generalizovaná budova. Pro nahrání nebo nakreslení jiného polygonu je zapotřebí spustit funkci *Clear all*. Po provedení generalizace se zobrazí dialogové okno s hodnocením efektivity (procento akceptovaných budov s $\Delta\sigma < 10°$).



Obrázek 3: *Ukázka Aplikace*

5 Tvorba spustitelného souboru

V souladu se zadáním byla vytvořena plně funkční a přeložená aplikace ve formě binárního souboru (`.exe`), která nevykazuje chyby. Tento spustitelný soubor pro operační systém Windows integruje veškeré nezbytné knihovny a datové složky, což umožňuje jeho okamžité spuštění bez nutnosti instalace interpretu jazyka Python.

K vytvoření binárního souboru byl využit nástroj *Auto-py-to-exe*, což je grafické rozhraní nad knihovnou *PyInstaller*. Vzhledem ke komplexnosti použitých knihoven (*PyQt6, GeoPandas, NumPy*) a přítomnosti externích grafických dat byl proces překladu optimalizován pomocí následujících kroků:

- **Izolace prostředí** – Pro minimalizaci velikosti výsledného souboru a zamezení konfliktů verzí bylo využito virtuální prostředí (venv) v rámci distribuce *Conda*, obsahující pouze nezbytné závislosti.
- **Ošetření cest k datům** – Jelikož *PyInstaller* při spuštění `.exe` souboru rozbaluje data do dočasného adresáře, byla do skriptu `MainForm.py` implementována funkce `resource_path(relative_path)`. Tato funkce dynamicky určuje absolutní cestu k ikonám, čímž předchází chybám při načítání grafiky.

- **Konfigurace sestavení** – Výsledný soubor byl sestaven s parametrem `-onefile` (všechna data v jednom souboru) a `-windowed`, který u aplikací s grafickým rozhraním potlačuje otevírání příkazového řádku na pozadí.

6 Dokumentace

6.1 Třída Algorithms (`algorithms.py`)

Třída `Algorithms` obsahuje logiku generalizace a geometrických výpočtů.

- `get2VectorsAngle(p1, p2, p3, p4)` – spočítá úhel mezi vektory $\vec{p_1p_2}$ a $\vec{p_3p_4}$ pomocí skalárního součinu.
- `removeDuplicatePoints(pol)` – odstraní duplicitní body z polygonu.
- `arePointsCollinear(pol)` – ověří, zda jsou všechny body kolineární pomocí cross productu.
- `createCHJ(pol)` – zkonstruuje konvexní obálku metodou Jarvis Scan s ošetřením singularit.
- `createCHG(pol)` – zkonstruuje konvexní obálku metodou Graham Scan s ošetřením singularit.
- `calculateDistance(p1, p2)` – spočítá euklidovskou vzdálenost dvou bodů.
- `getPointLinePosition(p, p1, p2)` – determinant pro test poloroviny.
- `createMMB(pol)` – spočítá min-max box a jeho plochu.
- `rotatePolygon(pol, sig)` – otočí polygon o daný úhel.
- `createMBR(building, use_graham)` – vytvoří minimální ohraničující obdélník pomocí opakované konstrukce MMB nad konvexní obálkou.
- `getArea(pol)` – spočítá plochu polygonu.
- `resizeRectangle(building, mbr)` – přeškáluje obdélník tak, aby měl stejnou plochu jako budova.
- `simplifyBuildingMBR(building, use_graham)` – generalizace metodou MBR.
- `simplifyBuildingPCA(building)` – generalizace metodou PCA.

- `simplifyBuildingLE(building)` – generalizace metodou Longest Edge.
- `simplifyBuildingWA(building)` – generalizace metodou Wall Average.
- `simplifyBuildingWB(building)` – generalizace metodou Weighted Bisector.
- `computeDeltaSigma(building, sigma)` – spočítá střední hodnotu čtverců úhlových odchylek pro hodnocení efektivity.

6.2 Třída Draw (`draw.py`)

Třída `Draw` zajišťuje veškeré grafické operace.

- `loadShapefile(file_name)` – načítá SHP soubor přes GeoPandas, transformuje souřadnice s uniformním měřítkem zachovávajícím poměr stran.
- `paintEvent(e)` – vykreslí budovy žlutou barvou a výsledné MBR červenou barvou.
- `mousePressEvent(e)` – při kliknutí přidá vrchol manuálního polygonu.
- `setMBR(mbr)` / `setMBRs(mbrs)` – nastaví výsledné obdélníky pro vykreslení.
- `getBuilding()` / `getBuildings()` – vrátí manuální polygon / seznam načtených budov.
- `clearResult()` – smaže výsledky generalizace.
- `clearAll()` – smaže veškerá data včetně budov.

6.3 Třída MainForm (`MainForm.py`)

Třída `MainForm` řídí inicializaci hlavního okna a propojuje třídy `Draw` a `Algorithms`.

- `simplifyBuildingMBRClick()` – generalizace všech budov metodou MBR, evaluace a zobrazení výsledků.
- `simplifyBuildingPCAClick()` – generalizace všech budov metodou PCA, evaluace a zobrazení výsledků.
- `simplifyBuildingLEClick()` – generalizace všech budov metodou Longest Edge, evaluace a zobrazení výsledků.
- `simplifyBuildingWAClick()` – generalizace všech budov metodou Wall Average, evaluace a zobrazení výsledků.
- `simplifyBuildingWBClick()` – generalizace všech budov metodou Weighted Bisector, evaluace a zobrazení výsledků.
- `openClick()` – otevře dialog pro výběr souboru a zavolá `loadShapefile`.
- `clearResultsClick()` – smaže výsledky generalizace.
- `clearAllClick()` – smaže veškerá data.

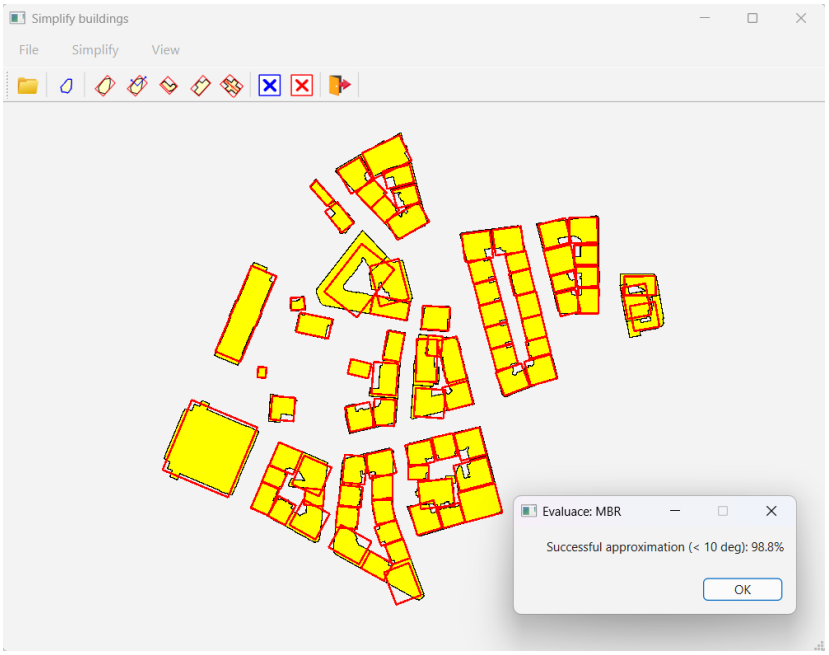
7 Výsledky

Efektivita detekce hlavních směrů budov byla vyhodnocena pomocí střední hodnoty čtverců úhlových odchylek (rovnice 10). V tabulce je uvedeno procento budov, pro které platí $\Delta\sigma < 10°$.

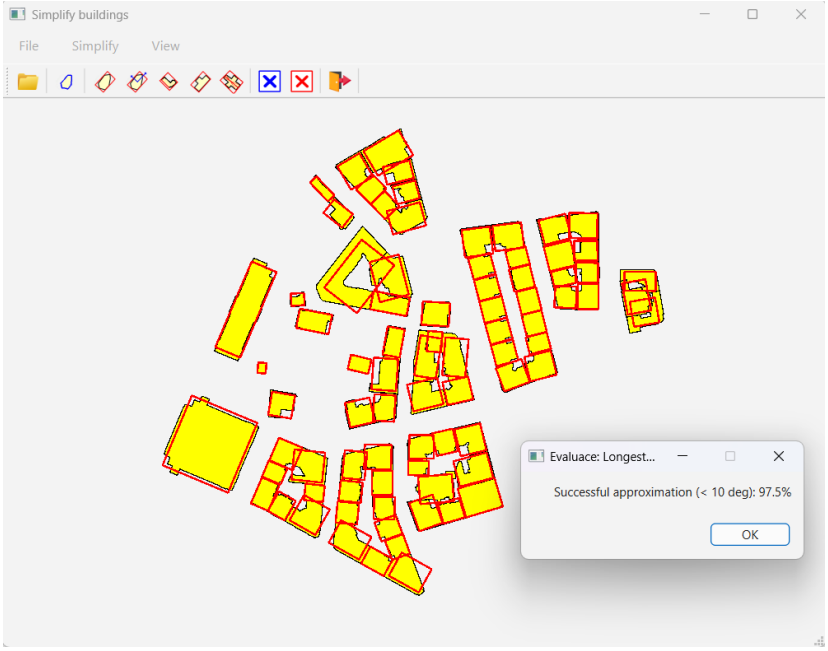
Katastrální území	**MBR**	**PCA**	**LE**	**WA**	**WB**
Josefov	98,8 %	66,7 %	97,5 %	93,8 %	58,0 %
Kročehlavy	100,0 %	92,6 %	100,0 %	100,0 %	71,8 %
Suchdol	95,5 %	55,7 %	95,5 %	89,8 %	56,8 %
průměr	98,1 %	71,7 %	97,7 %	94,5 %	62,2 %

Tabulka 1: Hodnocení efektivity generalizačních metod ($\Delta\sigma < 10°$)

Nejlepšího hodnocení dosáhl algoritmus **Minimum Area Bounding Rectangle** s průměrnou úspěšností 98,1 %. Metoda poskytuje celkově velmi dobré výsledky pro všechny typy zástavby. Velmi podobných výsledků dosáhla i metoda **Longest Edge** (97,7 %), která se vizuálně od MBR výslednou generalizací budov liší pouze nepatrně.



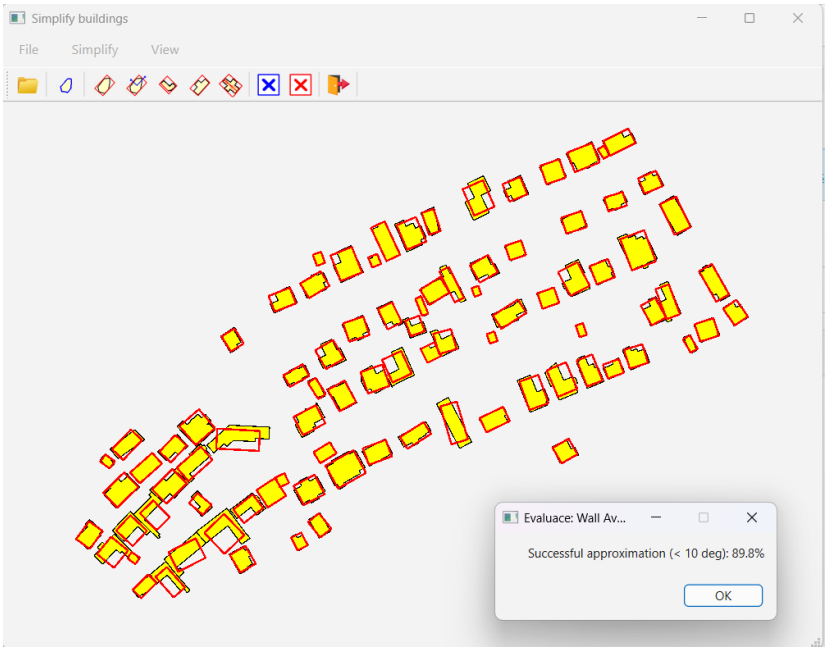
Obrázek 4: *Ukázka metody MBR pro historickou zástavbu Josefova (úspěšnost 98,8 %)*



Obrázek 5: *Výsledek metody Longest Edge pro lokalitu Josefova*

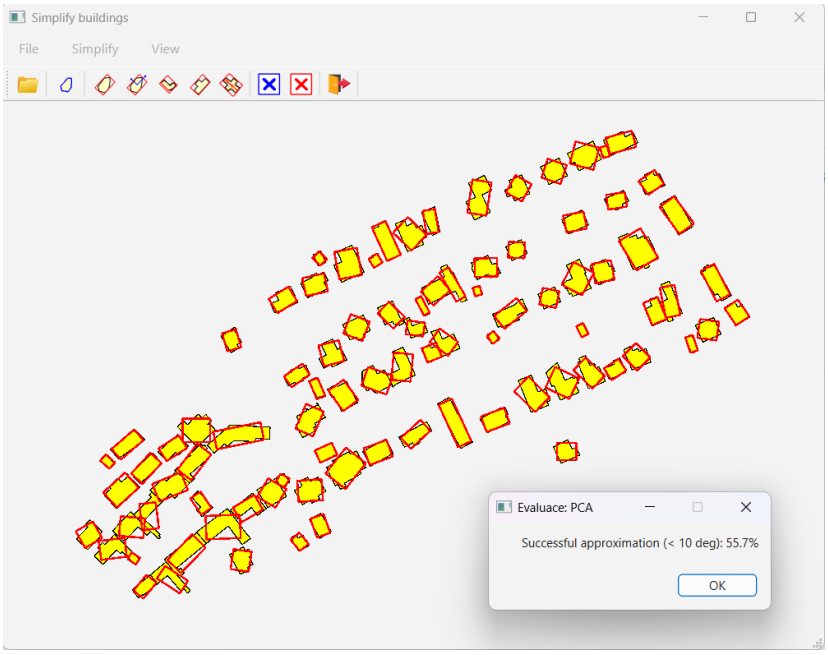
Metoda **Wall Average** dosáhla rovněž dobrých výsledků (94,5 %), přičemž u pravoúhlé sídlištní zástavby Kročehlav dosáhla 100 % úspěšnosti. U izolované zástavby Suchdolu s nepravidelnějšími půdorysy klesla na 89,8 %.

Obrázek 6: *Výsledek metody Wall Average na sídlišti Kročehlavy (úspěšnost 100 %)*



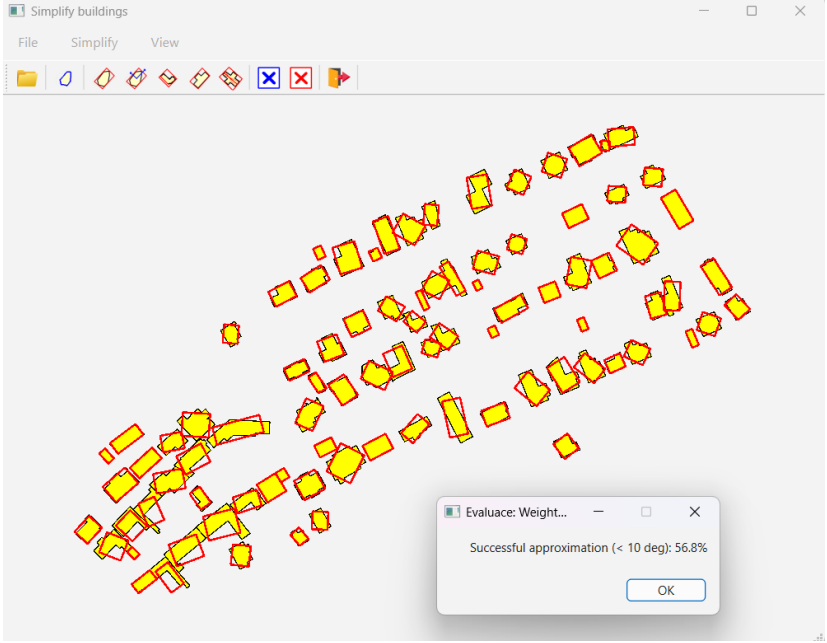
Obrázek 7: *Generalizace metodou Wall Average v lokalitě Suchdol*

Metoda **PCA** dosáhla výrazně horších výsledků (71,7 %). Při její aplikaci dochází k nevhodnému natočení budov, které mají výstupky nebo nepravidelný tvar, což je patrné zejména u Suchdolu (55,7 %).



Obrázek 8: *Výsledek při použití metody PCA na členité budovy v Suchdole*

Nejhorších výsledků dosáhla metoda **Weighted Bisector** (62,2 %). Problém spočívá v tom, že dvě nejdelší úhlopříčky nemusí vždy reprezentovat hlavní směr budovy, zejména u složitějších půdorysů.



Obrázek 9: *Výsledek metody Weighted Bisector pro lokalitu Suchdol (úspěšnost 56,8 %)*

8 Závěr

V rámci řešení problému generalizace budov LOD0 byly implementovány algoritmy Minimum Area Bounding Rectangle, Principal Component Analysis, Longest Edge, Wall Average a Weighted Bisector. Pro konstrukci konvexní obálky byly implementovány metody Jarvis Scan a Graham Scan s ošetřením singulárních případů (duplicitní body, kolineární body, méně než 3 body, shodný minimální y u pivotu). Vstupní data jsou načítána ze souboru shapefile s transformací souřadnic zachovávající poměr stran.

Jako možné vylepšení skriptu by byla možnost přibližování a pohybování se v okně aplikace, aby bylo možné si prohlédnout detailní rozdíl jednotlivých metod generalizace od vstupních budov.

Zdroje

[1] BAYER, T. (2026): *Konvexní obálka množiny bodů.* Přednáška pro předmět Algoritmy počítačové kartografie, Katedra aplikované geoinformatiky a kartografie, Přírodovědecká fakulta UK.

[2] ČÚZK (2026): *Katastrální mapa.* Český úřad zeměměřický a katastrální. `https://www.cuzk.cz/`

[3] ERICKSON, J. (2008): *Computational Geometry – Lecture 1: Convex Hulls.* University of Illinois at Urbana-Champaign. Dostupné z: `http://jeffe.cs.illinois.edu/teaching/compgeom/2008/notes/01-convexhull.pdf`